Pi Consulting

API de datos EPRE

Energía San Juan · Naturgy

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Desarrollado por

Sobre este documento: versión 0.1 · 29/10/2026 · Borrador para revisión.

Copyright © Pi Consulting 2026

Índice

Control de versiones 4
Registro de equipo de desarrollo de proyecto 4
Introducción 5
Alcance y responsabilidades 5
Información general del proyecto 6
Objetivos 6
Contexto organizacional 6
Descripción general del producto o resultado del proyecto 6
Información técnica del proyecto (DE) 7
Arquitectura: 7
Pipeline de ingesta cada 10 minutos: 7
Pipeline de ingesta cada 12 horas: 8
Pipeline de limpieza de archivos delta antiguos 8
Orígenes de datos: 10
ETL de la data 13
Introduccion 13
Bronze, Silver y Gold 13
ETL de los datos 14
Seguridad: 16
Información funcional del proyecto (API) 17
Finalidad y funcionamiento 17
Arquitectura del backend 17
Tecnologías y organización del código 18
Links de acceso al producto 18
Autenticación y solicitud HTTP 19
Ejemplo de uso en Postman 19
Estructura de la respuesta pública 19
Ejemplo de formato JSON 20
Reglas y límites del contrato 20
Respuestas, errores y tiempos de espera 21
Headers de respuesta y trazabilidad 22
Pruebas y estado de validación 22
Información del despliegue en Azure 23

Ambiente y arquitectura 23
Inventario de recursos 24
Runtime, capacidad y almacenamiento 24
Seguridad e identidad administrada 24
Autenticación por conexión 25
Variables y configuración 25
Infraestructura como código 26
Preparación local 27
Validación y creación de infraestructura 27
Publicación del código 27
Comprobación posterior y recuperación 27
Application Insights, logs y trazabilidad 28
Configuración de monitoreo 28
Consultas operativas 29
Información de soporte del proyecto 30
Procedimiento de atención 30
Preguntas frecuentes 30
Errores comunes o conocidos 31
Observaciones y pendientes de cierre 31
Términos comunes 32
Referencias del backend y Azure 32

Control de versiones

Versión	Descripción del cambio	Aprobado por	Fecha
0.1	Adaptación de la plantilla corporativa para la API EPRE y el despliegue Azure. Apartado DE reservado al analista.	Pendiente de revisión	29/10/2026

Registro de equipo de desarrollo de proyecto

Nombre	Rol	Empresa	Contacto
Joaquín Marsan	Backend, API y despliegue Azure	Pi Consulting	A confirmar para la entrega
A completar por el equipo	Ingeniería de Datos / analista responsable de DE	A confirmar	A completar
A confirmar	Coordinación y aprobación del proyecto	A confirmar	A completar
A confirmar	Referente técnico del cliente	Naturgy / Energía San Juan	A completar

El registro debe completarse y validarse con el equipo vigente antes de aprobar esta documentación.

Introducción

Este documento describe la API de datos EPRE de Energía San Juan y su implementación en Azure. Está dirigido a desarrolladores, administradores y equipos de soporte que deban consumir, publicar o mantener el servicio.

- Explicar la finalidad de la API, su contrato HTTP y la forma de consumirla.
- Documentar el backend, sus controles de seguridad, manejo de errores y pruebas.
- Identificar los recursos Azure, la configuración y el procedimiento de despliegue.
- Facilitar el diagnóstico de solicitudes mediante logs y trazabilidad.

La información técnica de Ingeniería de Datos se mantiene en un apartado independiente, a cargo del analista responsable. Los capítulos de API y Azure describen la fuente de datos como un servicio externo y no desarrollan sus procesos internos, cálculos ni administración.

Estado del documento. Borrador técnico versión 0.1, con revisión de código y entorno de desarrollo al 29/10/2026. La aprobación del contrato público y la prueba completa de integración desde Azure permanecen pendientes.

Alcance y responsabilidades

Responsable	Alcance
Backend / API	Contrato HTTP, autenticación de entrada, cliente del servicio de datos, validación, JSON público, errores y pruebas.
Azure / operación	Infraestructura, identidad del servicio, configuración, publicación y monitoreo.
Equipo DE	Contenido del apartado DE: preparación y significado de los datos, cálculos y servicio de origen.
Consumidor / aprobación	Validación del contrato de salida, necesidades de volumen y criterios de aceptación.

Información general del proyecto

Objetivos

Proveer un servicio HTTP de lectura que permita a los consumidores autorizados obtener información de EPRE en formato JSON, a través de un único endpoint administrado en Azure Functions.

- Publicar un punto de acceso estable para el consumo de información.
- Ejecutar las consultas con una identidad de servicio propia en Azure.
- Entregar respuestas controladas ante fallos de configuración, autenticación o datos.
- Registrar tiempos, estados e identificadores de correlación para soporte.

Contexto organizacional

La solución forma parte del trabajo de integración de información para Energía San Juan / Naturgy y sus consumidores EPRE. El equipo de Ingeniería de Datos define y mantiene la información disponible; el backend ofrece el acceso HTTP y la operación del servicio en Azure.

Los cálculos de indicadores y la preparación de las fuentes pertenecen al alcance DE. La API selecciona los campos acordados y entrega las colecciones recibidas; no calcula indicadores ni incluye una interfaz de visualización.

Descripción general del producto o resultado del proyecto

El producto documentado es una Azure Function Python con trigger HTTP GET, desplegada en un entorno de desarrollo de Azure. Recibe una solicitud autenticada, consulta el servicio de datos, valida la respuesta y devuelve JSON o un error controlado.

Aspecto	Implementación actual
Consumidores	EPRE y herramientas de prueba, por ejemplo Postman.
Operación publicada	GET /api/datos/epre. Una solicitud válida produce una consulta al servicio de datos.
Información expuesta	Nueve colecciones de KPI, demanda/generación, mensajes, reclamos e instalaciones afectadas, incluyendo las colecciones GPS.
Persistencia de negocio	El backend no almacena las filas recibidas ni mantiene un histórico propio.
Contrato	Provisional, a validar con el consumidor y el equipo DE.
Ambiente	Desarrollo. Este documento no acredita una liberación productiva.

Información técnica del proyecto (DE)

Apartado a cargo del analista de Ingeniería de Datos. Se conserva el contenido recibido en la plantilla para su revisión y actualización por ese responsable. No se valida ni se amplía en esta entrega de API y Azure.

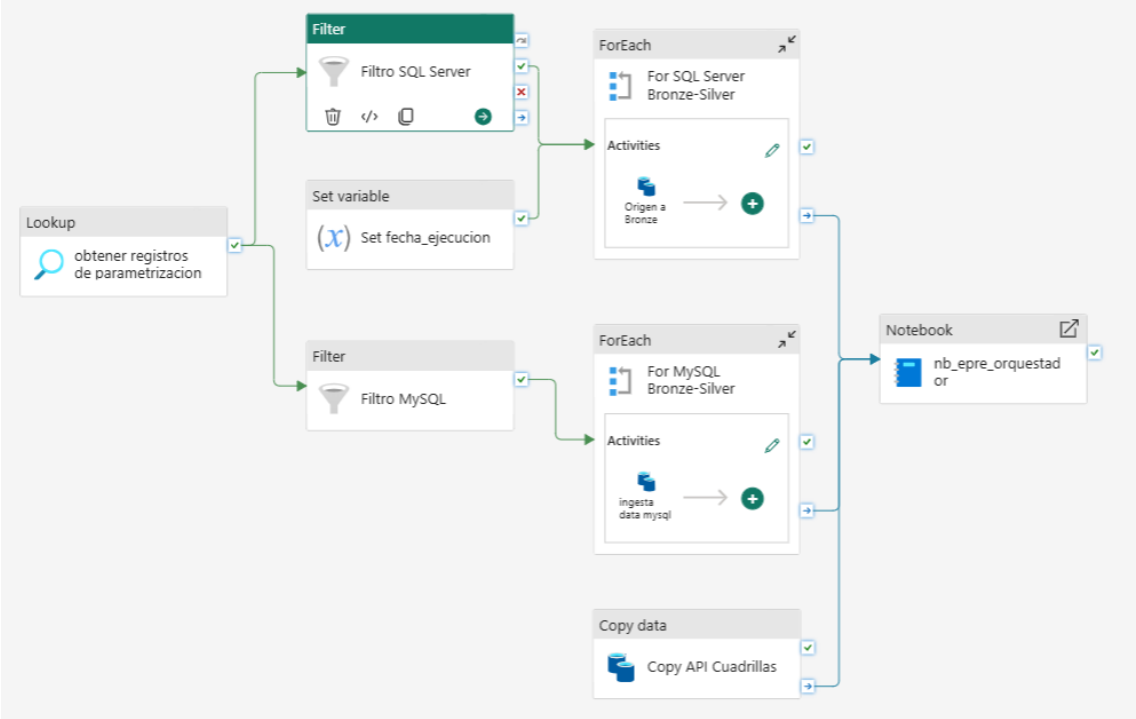
Arquitectura:

La arquitectura propuesta para este sistema se describe a continuación:

El diseño final responde a una versión simplificada de un framework de arquitectura de Pi Consulting desarrollado por Santos Oliva, en el que los orígenes de datos y transformaciones se almacenan en tablas de management con sus respectivos metadatos. El inconveniente que surgió durante el desarrollo fue la suma de los pequeños tiempos de cada uno de esos procesos, no permiten la actualización del pipeline en los tiempos que la necesidad del cliente requiere, que es de 5 minutos. Al momento de redactar este documento, el pipeline actualiza cada 10 minutos

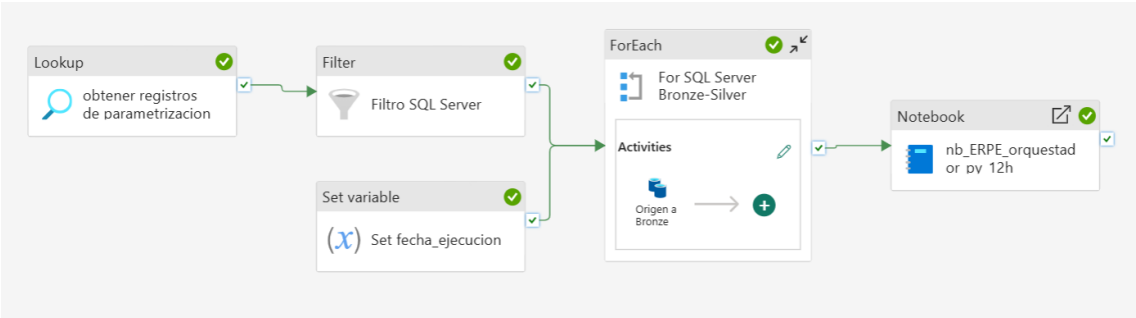
Finalmente, esta simplificación de la arquitectura nos lleva a los siguientes pipelines:

Pipeline de ingesta cada 10 minutos:



Link al pipeline

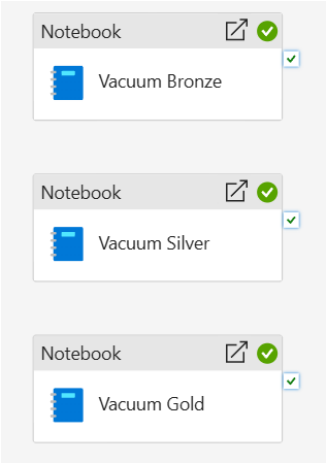
Pipeline de ingesta cada 12 horas:



Link al pipeline

El primer pipeline contiene tres origenes de datos, una API con la informacion de las cuadrillas, una base de datos MySQL con los datos de mensajes de la aplicacion que el cliente llama Cartero, y una base de datos SQL Server, con las tablas que debemos ingestar cada 10 minutos, el segundo pipeline, mantiene la logica del primero en la ingesta de las tablas, solo que se ejecuta cada 12 horas, y solo toma datos de una tabla SQL Server, no hay ingestas de la API y de MySQL en este pipeline. En el apartado ETL de la data, en este mismo documento, se explican en detalle estos procesos.

Pipeline de limpieza de archivos delta antiguos



Este pipeline se ejecuta de manera semanal y realiza la limpieza de archivos Delta antiguos en los lakehouses correspondientes mediante procesos de *vacuum*. Su función es eliminar archivos físicos que ya no son referenciados por las versiones vigentes de las tablas Delta, liberando espacio de almacenamiento y manteniendo ordenada la estructura interna de los datos.

Este proceso es necesario porque las tablas Delta conservan archivos históricos para permitir operaciones como versionado, recuperación y consistencia transaccional.
La ejecución periódica del *vacuum* contribuye a optimizar costos, mejorar la administración del lakehouse y reducir la acumulación de archivos obsoletos.

Link al pipeline

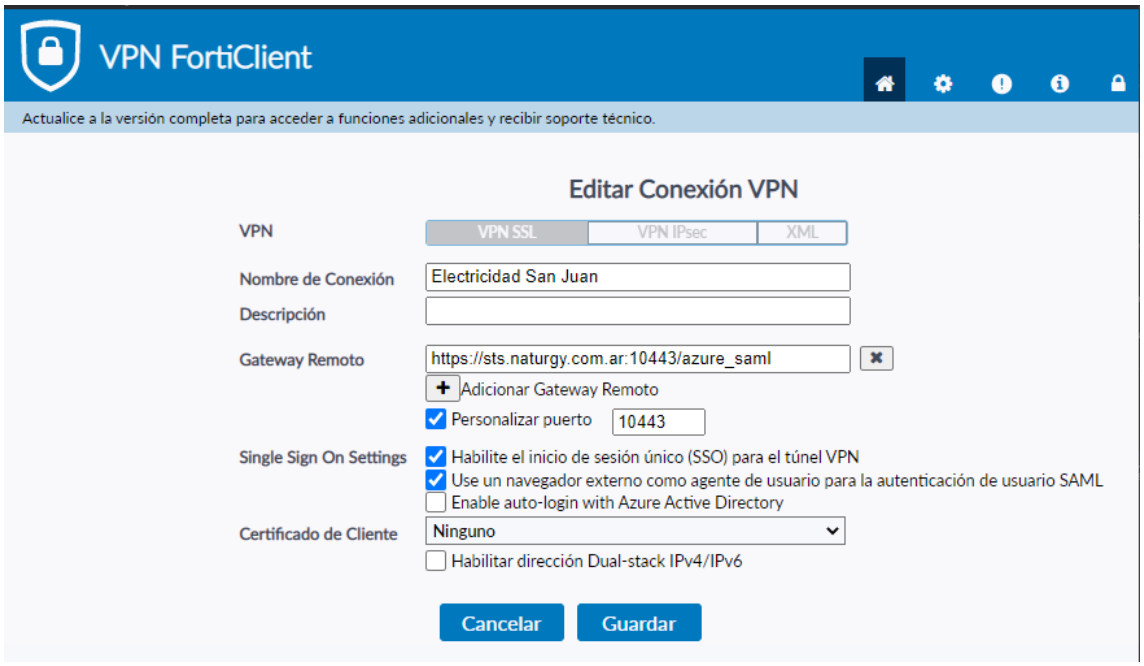
Orígenes de datos:

Todas las conexiones a los datos, se hicieron mediante un Gateway instalado exclusivamente para este proyecto, instalado en una maquina virtual provista por el cliente dentro de su red. Los datos de las distintas conexiones son los siguientes:

Conexion a VPN:
Mediante Forti Client

Direccion: https://sts.naturgy.com.ar:10443/azure_saml

Puerto: 10443



Por supuesto, que el usuario deberá tener sus credenciales aprobadas en la red.

Con la VPN se accede a la maquina virtual, que tiene la aplicacion de Gateway que permite vincular la red de Electricidad San Juan con Azure Fabric

Para acceder a la Maquina virtual, se utilizan las siguientes credenciales:

IP VM: 192.9.206.101

Usuario: energiasanjuan\99011646

Contraseña: Tab******

Dentro de esta maquina virtual, esta instalado el Gateawy que nos permite conectarnos a todos los origenes de datos.

Todos estos datos fueron provistos por Sergio Castillo, de Naturgy.

A continuación, se enumeran los diferentes orígenes de datos utilizados para la ejecución de este proyecto:

- SQL Server, accediendo desde Fabric mediante el gateway GatewayCartero y la conexion Sql_server_129.1.215.26_prd (Conector SQL Server nativo de Fabric)
  Nombre: SUR
  Formato: SQL Server 2019
  IP: 129.1.215.26
  Base de Datos: reportes
  Usuario: piconsulting
  Password: 0k******
  Tiene permiso de lectura sobre todas las tablas

  De este conector, se toman las siguientes tablas:

  demandascada
  GEOSETAS
  MULTASETASAFECTADAS
  RECLAMOSDERIVADOS
  RECLAMOSPASADOS
  RECLAMOSSINPASAR
  SETASAFECTADAS
  USUARIOSAFECVSTOTALES
  USUARIOSLOGUEADOS

  Estas configuraciones fueron provistas por Luis Alejando Rodrigo (lrodrigo@naturgy.com.ar) de Naturgy, y la actualizacion de la tabla demandascada será realizada por Leopoldo Iribarren (liribarren@naturgy.com.ar)

- MySQL, accediendo desde Fabric mediante el gateway GatewayCartero y la conexion Cartero (Conexion ODBC en Fabric)
  Formato: MySQL
  IP: 192.9.202.5
  Base de Datos: cartero_v2
  Usuario: conexion_fabric
  Password: fa*******

  De este conector, se toman la siguiente tabla:

  vw_bi_mensajes_cartero

  Estas configuraciones fueron provistas por Sergio Castillo de Naturgy

- API Rest, accediendo desde Fabric mediante el gateway GatewayCartero y la conexion API Cuadrillas GW (Conexion de API REST nativa de Fabric)
  Formato: API REST

URL:
https://centrality8.energiasanjuan.com.ar/backend/esj/svc_esj_published_services.php?action=ubicacion_cuadrillas
Usuario: ps_ubicacion_cuadrillas
Password: ps*****

De este conector, se toman la siguiente tabla:

API_Cuadrillas

Estas configuraciones fueron provistas por Marcelo Javier Diaz (mdiaz@naturgy.com.ar) de Naturgy

ETL de la data

Introduccion

Para lograr la mejor performance en los tiempos de ejecucion, se decidio prescindir de notebooks de procesamiento en Python-Spark, y en lugar de eso, se utilizaron librerías de Python nativas, para evitar los tiempos de inicio y ejecución de los motores de Spark, además, de todas las opciones evaluadas, fue el proceso mas barato en uso de recursos.

Los datos se alojan en tres esquemas dentro de tres Lakehouse, uno para cada etapa, LH_Bronze, LH_Silver, LH_Gold, las ingestas y orquestación se hacen mediante Fabric Pipeline, y las transformaciones mediante Notebooks Python.

Bronze, Silver y Gold

Los datos viven en tres capas, Bronze, Silver, y Gold:

Los datos en Bronze son copias fieles de los datos de origen, permiten ingresar al ecosisstema cloud, e independizarse del origen de datos, los datos llegan sin alteraciones.

Los datos en Silver, por otro lado, enriquecen los datos de origen, acomodan formatos, y renombran columnas, la idea de esta capa es dejar datos listos para ser procesados por las areas de negocio, pero aun no aportan valor.

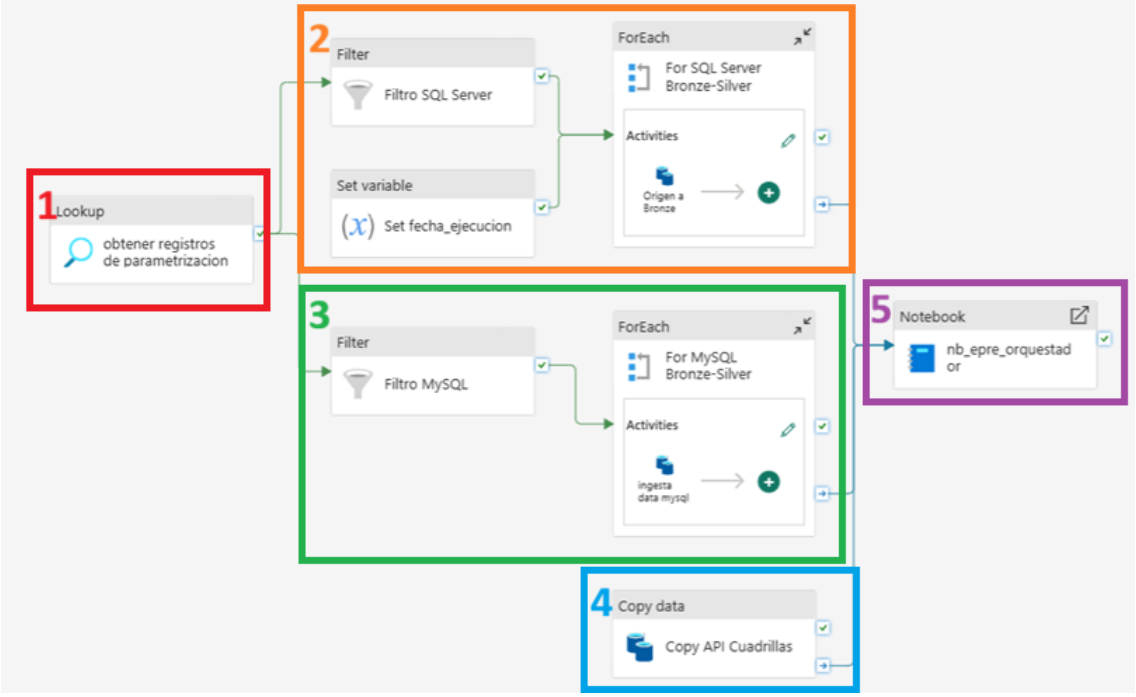
Se corrigen por ejemplo, datos de fecha que llegaron como texto, nombres de columnas que no explican que dato quieren mostrar, se generan tablas auxiliares (por ejemplo, cada vez que se ingesta una tabla de reclamos, se guarda ademas de esa tabla, la cantidad de esos registros en otra tabla).

Los datos en Gold, finalmente, son los datos, tablas y medidas utiles para el negocio, son las tablas finales que consumen los reportes de Power BI. Por ejemplo, en Silver tenemos la tabla de Setas afectadas, y la tabla con las coordenadas de latitud y longitud de las mismas, mientras que en gold, tenemos esas tablas ya cruzadas, y tenemos en la misma tabla, las Setas afectadas y su ubicacion GPS.

Con esta breve introduccion, pasamos a la explicacion del proceso completo de ingesta.

ETL de los datos

Para explicar detalladamente el proceso, dividimos el pipeline de ingesta que se ejecuta cada 10 minutos en 5 bloques:



1. Obtener registros de parametrizacion
   La lista de tablas a extraer, y sus metadatos relacionados, necesarios para el proceso, se guardan en un Lakehouse de Management, lo que nos permite facilmente agregar o eliminar nuevos origenes de datos

   Esta es una captura de la tabla de management de origenes, donde se omiten columnas que no son utilizadas en este proyecto:

	activo	frecuencia	origen	nombre_tabla	ubicacion_relativa	Tipo_carga
1	1	5m	mysql	vw_bi_mensajes_cartero	NULL	full
2	1	5m	sql_server	USUARIOSAFECVSTOTALES	dbo	full
3	1	5m	sql_server	MULTASETASAFECTADAS	dbo	full
4	1	12h	sql_server	RECLAMOSDERIVADOS	dbo	full
5	1	5m	sql_server	USUARIOSLOGUEADOS	dbo	full
6	1	5m	sql_server	RECLAMOSSINPASAR	dbo	full
7	1	5m	sql_server	RECLAMOSPASADOS	dbo	full
8	1	5m	sql_server	SETASAFECTADAS	dbo	full
9	1	5m	sql_server	GEOSETAS	dbo	full
10	1	5m	sql_server	demandascada	dbo	full

Se observa el estado activo, la frecuencia de actualizacion (5m: 5 minutos, 12h: 12 horas), el origen, el nombre de la tabla en origen, su ubicación relativa en origen, estos datos son utilizados para realizar la Ingesta de SQL Server y MySQL.

Una vez tenemos la tabla de metadatos, filtramos por condiciones, en este caso, que esté activo el origen, que sea SQL Server, y que ingeste cada 5 minutos.

Con esos tres datos, tenemos la lista reducida, y mediante la actividad de copia Origin a Bronze, guardamos la tabla en nuestro Lakehouse Bronze en Fabric, para luego armar las transformaciones a Silver y Gold, este proceso se realiza en loop paralelo, es decir, se hacen todas estas operaciones de copia a la vez.

2. Ingesta de MySQL
   Con filtros similares al proceso 2 excepto que en lugar de SQL Server filtramos por MySQL, tomamos las tablas de ese origen, las ejecutamos en paralelo (al momento de hacer este documento, solo es una tabla), y luego se procede con el Notebook Python de transformación.
3. Ingesta de API Rest
   En este caso, al ser una sola API especifica, en lugar de parametrizarla, se carga manualmente su Copy Data y se guarda en Bronze, similar a los demás procesos.

Como dato interesante, se ve que estas tres etapas son exactamente iguales en logica, eso nos permite a futuro la escalabilidad y agregar nuevas tablas, u origenes, sin tener que reprocesar o refactorizar el proceso.

4. Finalmente, una vez que se ejecutaron correctamente los passos anteriores, se ejecuta el Notebook Python que crea las tablas de Silver y Gold,
   Notebook Transformacion Silver y Gold

No hemos mencionado el segundo pipeline que mostramos al principio, el que se ejecuta cada 12hs, porque ese es literralmente las etapas 1 y 2 aqui explicadas, pero ejecutadas cada 12hs, otra muestra de lo flexible que es la arquitectura desarrollada.

Seguridad:

En este proyecto utilizamos las siguientes funcionalidades para mantener la seguridad del mismo:

- Autenticación centralizada mediante Azure Active Directory: todos los usuarios acceden a la solución utilizando sus credenciales corporativas, permitiendo validar su identidad desde el directorio central de la organización.
- Asignación de permisos por usuario o grupo de seguridad: los accesos pueden otorgarse de forma individual o, preferentemente, mediante grupos de seguridad, facilitando la administración de altas, bajas y cambios de permisos.
- Control de acceso a los workspaces de Microsoft Fabric y Power BI: el acceso a los elementos del proyecto se limita según los permisos asignados en las áreas de trabajo, evitando que usuarios no autorizados visualicen o modifiquen componentes técnicos o reportes.
- Uso de gateway corporativo para conexión segura con orígenes de datos alojados en la red de Energía San Juan: las conexiones hacia SQL Server, MySQL y API REST se realizan a través del gateway instalado en infraestructura del cliente, permitiendo acceder a los datos internos sin exponer directamente los sistemas de origen.
- Gestión de credenciales de conexión desde las configuraciones administradas de Fabric: las credenciales utilizadas por los conectores se administran desde las conexiones configuradas en Fabric y el gateway, evitando incluir contraseñas en reportes, modelos semánticos, notebooks o documentación funcional.
- Uso de Plataforma PiBi para acceder a los reportes, que asegura el consumo de usuarios internos y externos a los reportes correspondientes

Información funcional del proyecto (API)

Finalidad y funcionamiento

La API permite solicitar la información de EPRE mediante un GET. El consumidor no envía una consulta de datos en el body: la consulta utilizada está incluida en el código y es mantenida por el equipo de desarrollo con la definición recibida de DE.

Cada solicitud válida consulta la fuente en ese momento. El backend no programa refrescos, no conserva resultados en caché y no agrega reglas de cálculo. La vigencia de la información corresponde al servicio de datos y a sus procesos de actualización.

Arquitectura del backend

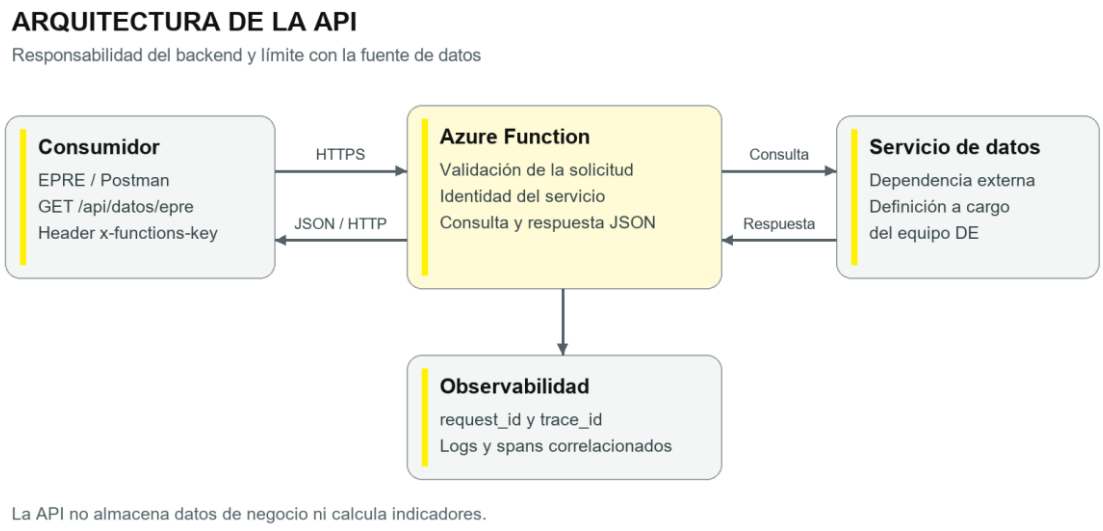


Figura 1. Recorrido de una solicitud y responsabilidad del backend.

Paso	Comportamiento
1. Entrada	Azure Functions valida la Function Key. El handler sólo admite GET y valida que no haya body ni filtros de datos.
2. Configuración	Se valida la URL configurada y la selección de credencial permitida para el ambiente.
3. Autenticación de servicio	El SDK obtiene un token. En Azure usa la identidad administrada del sistema; en local, Azure CLI.
4. Consulta	El cliente realiza un POST HTTPS al servicio externo, con la consulta fija y el contexto de trazabilidad.
5. Validación y salida	Se validan las nueve colecciones y sus campos; se seleccionan los campos públicos y se devuelve JSON.
6. Diagnóstico	Se registran inicio, fin, duración, estado y cantidades. Ante un fallo se devuelve un código público controlado.

Tecnologías y organización del código

Se utiliza el modelo de programación Python v2 de Azure Functions, con decoradores y archivo function_app.py. Este modelo es independiente del número del runtime Functions: el despliegue utiliza Functions 4 sobre Linux con Python 3.12.

Dependencia	Versión fijada
azure-functions	1.25.0
azure-identity	1.26.0
requests	2.34.2
azure-monitor-opentelemetry-exporter	1.0.0b58
opentelemetry-api	1.45.1
opentelemetry-sdk	1.45.1

El exportador azure-monitor-opentelemetry-exporter está fijado en una versión prerelease. La publicación instala las dependencias de requirements.txt mediante build remoto Linux.

Archivo o directorio	Responsabilidad
function_app.py	Trigger GET, validación HTTP, respuestas JSON, headers de correlación y traducción de errores.
esj_api/config.py	Lectura y validación del endpoint y de la credencial habilitada.
esj_api/fabric.py	Cliente del servicio de datos: token, POST HTTPS, timeouts y validación de la respuesta. El nombre del módulo refleja la integración existente.
esj_api/queries/epre.graphql	Consulta fija entregada por DE. Su definición funcional se coordina con ese equipo.
esj_api/transform.py	Selección de campos y armado del contrato público por colección.
esj_api/telemetry.py	Contexto por solicitud, logs JSON y spans manuales.
host.json	Ruta base, timeout y configuración de logs/telemetría del host.
tests/	Pruebas automatizadas del contrato, fallos, privacidad, correlación y empaquetado.
scripts/	Diagnóstico local/remoto y creación del ZIP de publicación.
deploy/	Plantilla ARM, publicación y consultas de telemetría.

Links de acceso al producto

Ambiente	Dirección
Desarrollo Azure	https://func-pi-epre-dev-hnso27.azurewebsites.net/api/datos/epre
Desarrollo local	http://localhost:7071/api/datos/epre
Método	GET
Ruta implementada	/api/datos/epre
Producción	No definida ni desplegada en el alcance verificado.

Las rutas de versiones iniciales, /api/datos/resumen y /api/datos/setas-afectadas, no forman parte de la implementación vigente.

Autenticación y solicitud HTTP

Elemento	Uso
x-functions-key	Obligatorio en Azure. Function Key obtenida por el canal de administración autorizado.
Accept: application/json	Opcional para expresar el formato esperado.
traceparent	Opcional. Contexto W3C para continuar una traza distribuida.
Body	Vacío. No enviar JSON ni la consulta del servicio de datos.
Params de datos	Vacíos. No se admiten desde, hasta, region ni otros filtros.

La aplicación permite el parámetro reservado code del host, pero la forma indicada de enviar la clave es el header x-functions-key. No incluir claves ni datos personales en la URL. La Function Key no se reenvía al servicio de datos.

Ejemplo de uso en Postman

- Crear una petición GET y copiar el endpoint de desarrollo Azure.
- En Headers agregar x-functions-key con una variable privada, por ejemplo {{function_key}}.
- Dejar Params vacío y seleccionar Body → none.
- Enviar la solicitud y registrar el estado HTTP, X-Request-ID y X-Trace-ID.

```
GET https://func-pi-epre-dev-hnso27.azurewebsites.net/api/datos/epre
x-functions-key: {{function_key}}
Accept: application/json
```

Core Tools local normalmente no exige la Function Key. Por eso la protección de entrada se valida también sobre la Function desplegada en Azure.

Estructura de la respuesta pública

El JSON devuelve origen y colecciones. Cada colección conserva el nombre acordado con el servicio externo y expone cantidad_devuelta, limite_solicitado e items. Los campos de las filas se seleccionan expresamente; sus valores y tipos se conservan tal como fueron recibidos.

Campo	Descripción
origen	Identificador de la integración. El valor actual es fabric.
colecciones	Objeto con las nueve colecciones públicas.
cantidad_devuelta	Número de elementos incluidos en items; no representa el total de la fuente.
limite_solicitado	null en la consulta actual porque no fija un límite explícito.
items	Lista de registros con los campos seleccionados. Puede estar vacía si así responde la fuente.

Colección pública	Campos expuestos
vw_kpi_epres	Metrica, Fecha, Hora, Valor
demanda_Generacions	Fecha_Hora, Demanda, Generacion
carteros	fecha_hora, mensaje
cantidad_Reclamos	Fecha_Hora, Reclamos_Totales
reclamos	Fecha_Hora, Descripcion, Reclamo, Suministro, Tipo
vw_reclamos_gps	Tipo, Latitud, Longitud, Suministro, Reclamo, Fecha_Hora, Descripcion
cantidad_Setas_Afectadas	Fecha_Hora, Cantidad
setas_Afectadas	Fecha_Hora, ET_Distribuidor, ID_Seta, Usuarios_Afectados
vw_setas_afectadas_gps	Tipo, Latitud, Longitud, ET_Distribuidor, Setas, Fecha_Hora, Descripcion

La definición de cada indicador, sus unidades, la semántica de fechas y el significado de los datos corresponde al apartado DE y a la validación del contrato con el consumidor.

Ejemplo de formato JSON

Fragmento ilustrativo con una colección y una lista vacía. La respuesta completa contiene las nueve colecciones de la tabla anterior. El ejemplo muestra el formato; no representa una consulta real ni acredita que la fuente esté vacía.

```
{
  "origen": "fabric",
  "colecciones": {
    "vw_kpi_epres": {
      "cantidad_devuelta": 0,
      "limite_solicitado": null,
      "items": []
    }
  }
}
```

Reglas y límites del contrato

- Se preservan nulos, duplicados, orden y tipos escalares recibidos. No se normalizan fechas ni se agrupan filas.
- Se devuelven únicamente los campos incluidos en la selección pública.
- Una colección o un campo obligatorio ausente produce error; no se reemplaza por datos inventados.
- Los errores de la consulta invalidan la respuesta completa, aun cuando el proveedor incluya datos parciales.
- No hay paginación, filtros ni orden explícito implementados. El tamaño recibido no garantiza que se haya descargado todo el conjunto disponible.

- No se calculan promedios, totales de negocio ni variaciones porcentuales dentro del backend.

Respuestas, errores y tiempos de espera

HTTP	Código público	Significado
200	—	Respuesta completa validada y JSON público generado.
400	solicitud_invalida	Body no vacío o parámetros de datos no admitidos.
401	Respuesta del host	Function Key ausente o inválida en Azure. Puede usar otro formato.
500	error_interno	Fallo inesperado, sin exponer detalles internos al consumidor.
502	error_proveedor	Conexión fallida, HTTP inesperado, JSON inválido, errores de consulta o estructura incompatible.
503	servicio_no_disponible	Configuración, token, rechazo de autorización del origen o indisponibilidad temporal.
504	tiempo_agotado	Timeout de conexión o lectura del servicio de datos.

Los estados externos 401, 403, 429 y 503 se traducen a 503. Un HTTP 404 del servicio externo o una respuesta HTTP 200 con errores de consulta se traduce a 502. Las validaciones y errores propios mantienen la misma envoltura JSON.

```
{
  "error": {
    "codigo": "servicio_no_disponible",
    "mensaje": "El servicio de datos no está disponible. Contactá al equipo técnico con el
request_id.",
    "request_id": "<identificador de la solicitud>"
  }
}
```

Control	Valor actual
Timeout de conexión HTTPS	5 segundos.
Timeout de lectura HTTPS	30 segundos.
Timeout configurado del host	2 minutos.
Reintentos automáticos del POST	No implementados.
Redirecciones HTTP	No se siguen.
Caché de respuestas	No implementada. La credencial sí se reutiliza en memoria por worker.

Headers de respuesta y trazabilidad

Header	Finalidad
Content-Type	application/json con codificación UTF-8 en las respuestas propias.
X-Request-ID	Identificador de la solicitud para informar al equipo de soporte.
X-Trace-ID	Identificador de la traza distribuida, usado como operation_Id en Application Insights.
Cache-Control	no-store. Evita almacenar las respuestas en cachés compatibles.

Los errores emitidos por la plataforma antes de ejecutar el código, como una clave inválida o una ruta inexistente, pueden no incluir los headers o la envoltura JSON de la aplicación.

Pruebas y estado de validación

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Al 29/10/2026 se ejecutaron 30 pruebas automatizadas con resultado correcto. Las pruebas controlan las respuestas externas y no necesitan consultar datos reales en cada ejecución.

Grupo	Cobertura
Contrato y datos	Nueve colecciones, campos, listas vacías, nulos, selección pública y rechazo de estructuras incompatibles.
Errores	Token, permisos, conexión, timeout, errores de consulta y datos parciales.
Telemetría	Correlación de logs y spans, trazas entrantes/salientes, aislamiento entre solicitudes y privacidad.
Configuración y paquete	Credenciales permitidas según ambiente y exclusión de settings, datos y entorno local del ZIP.

Prueba sobre Azure	Resultado observado
Sin clave	401
Clave inválida	401
Clave válida y parámetro no admitido	400
Clave válida y consulta de datos	503
Correlación en respuestas propias	X-Trace-ID coincide con la traza enviada; X-Request-ID presente.

Integración pendiente. La prueba de datos realizada el 29/10/2026 respondió HTTP 503. La Function está publicada, pero no se acredita todavía un HTTP 200 con datos usando su identidad en Azure. El cierre de integración requiere resolver el acceso al servicio de origen y repetir la prueba. Los detalles de administración de esa fuente corresponden al equipo DE.

Información del despliegue en Azure

Ambiente y arquitectura

La infraestructura de desarrollo se creó el 27/10/2026 mediante una plantilla Azure Resource Manager (ARM). El código se publica sobre una Function App Linux en Flex Consumption, con almacenamiento para el host y los paquetes y una integración de observabilidad basada en Application Insights y Log Analytics.

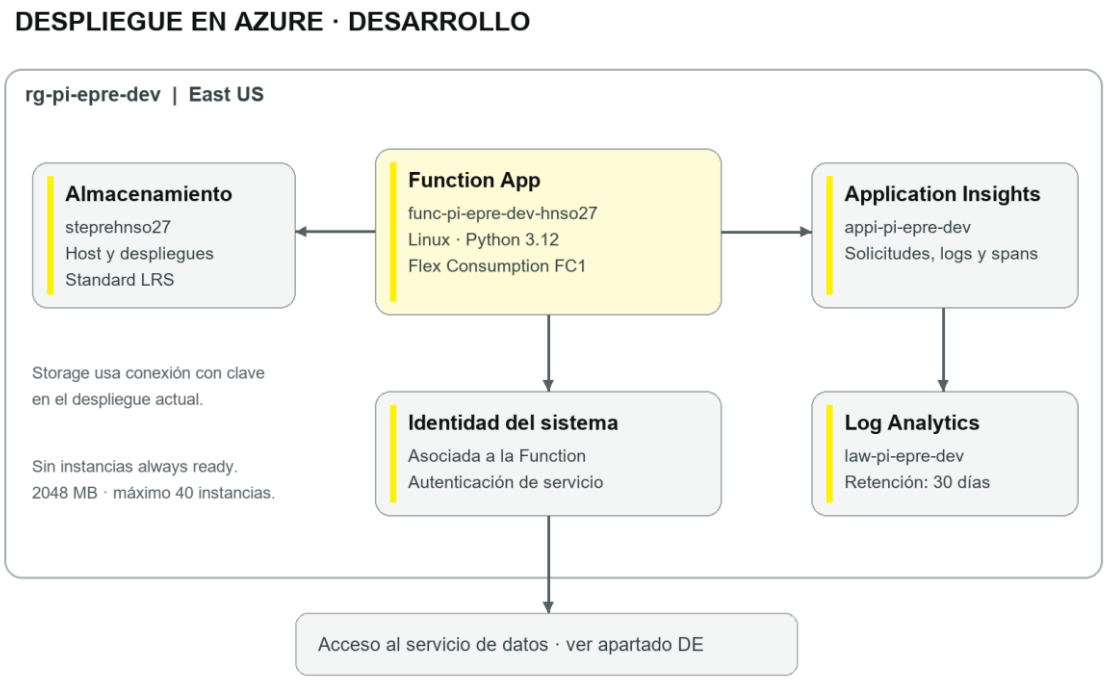


Figura 2. Recursos y conexiones del entorno Azure de desarrollo.

Dato del ambiente	Valor
Suscripción	Azure subscription 4
Subscription ID	d468911c-54a0-4a0b-af8c-938d81aeea30
Directorio	Naturgy Argentina
Tenant ID	cddb6514-1d4f-4a4a-a3ed-a1ca3de10829
Grupo de recursos	rg-pi-epre-dev
Región	East US / eastus
Fecha de creación	27/10/2026
Revisión del backend y la Function	29/10/2026

Inventario de recursos

Recurso	Nombre	Función
Function App	func-pi-epre-dev-hnso27	Ejecuta el endpoint HTTP y el backend Python.
Plan FC1	asp-pi-epre-dev	Flex Consumption sobre Linux.
Storage Account	steprehnso27	Estado interno del host y contenedor privado de despliegues. No es una base de datos de negocio.
Application Insights	appi-pi-epre-dev	Solicitudes del host, logs y spans instrumentados.
Log Analytics	law-pi-epre-dev	Workspace asociado para consulta y retención de telemetría.
Identidad del sistema	func-pi-epre-dev-hnso27	Identidad vinculada al ciclo de vida de la Function App.

La plantilla no incorpora API Management, un servidor de base de datos del backend ni un balanceador adicional. La entrada HTTP es la dirección HTTPS administrada por Azure Functions.

Runtime, capacidad y almacenamiento

Parámetro	Configuración
Sistema / lenguaje	Linux / Python 3.12. Modelo de programación Python v2.
Plan	Flex Consumption, SKU FC1.
Memoria por instancia	2048 MB.
Máximo de instancias	40. Es un límite de escalado, no una cantidad de instancias permanentemente activas.
Always ready	Sin instancias configuradas.
Timeout del host	00:02:00, definido en host.json.
Storage	StorageV2, Standard_LRS, contenedor deployments sin acceso anónimo.
Transporte	HTTPS obligatorio, TLS mínimo 1.2. FTP deshabilitado.

El entorno escala según la demanda del plan. La configuración actual no establece un SLA propio del producto, una prueba de capacidad ni un presupuesto monetario. El consumo de cómputo, almacenamiento y telemetría debe revisarse con el uso real antes de producción.

Seguridad e identidad administrada

La Function tiene una identidad administrada asignada por el sistema. Azure crea su representación en Microsoft Entra y la asocia al recurso. La aplicación desplegada selecciona explícitamente ManagedIdentityCredential para obtener el token utilizado al consultar el servicio externo.

Identificador	Valor
Nombre	func-pi-epre-dev-hnso27
Tipo	SystemAssigned / ManagedIdentity
Object ID / Principal ID	076d1b24-1406-43bc-a477-922d03cb5cc0
Application / Client ID	40aeffea-62db-4902-bf97-b5c607b653ad
Tenant ID	cddb6514-1d4f-4a4a-a3ed-a1ca3de10829

El Principal ID identifica el objeto de la identidad en el directorio. El Application ID es otro identificador de esa misma representación de servicio; no es la clave HTTP del consumidor. La habilitación de la identidad en Azure no implica que todas las dependencias la hayan autorizado.

Autenticación por conexión

Conexión	Mecanismo actual
Consumidor → Function	Function Key validada por el host. El cliente la envía mediante x-functions-key.
Function → servicio de datos	Identidad administrada del sistema. La autorización sobre la fuente se gestiona con el responsable DE.
Function → Storage	Cadena de conexión con la clave de la cuenta, generada dentro de ARM y guardada en los app settings.
Host / spans → Insights	Connection string del recurso asociado. El host exporta logs y solicitudes; el worker exporta spans manuales.
Publicación	Azure CLI autenticado con Microsoft Entra. Credenciales básicas de SCM y FTP deshabilitadas por la plantilla.

La identidad administrada se utiliza para el servicio de datos; el almacenamiento del host y del despliegue todavía utiliza una conexión con clave. Migrar también esa conexión a identidad requiere conceder previamente los roles Azure de datos correspondientes y modificar la plantilla. No se documenta esa migración como implementada.

Los endpoints de red de la Function y Storage son públicos y requieren autenticación. No se configuraron red virtual, Private Endpoints ni restricciones de red privadas en la infraestructura actual.

Variables y configuración

La configuración local y la de Azure se mantienen separadas. local.settings.json no se publica. En Flex Consumption, el runtime y el contenedor de despliegue se configuran en functionAppConfig de la plantilla ARM.

Variable / propiedad	Valor o propósito
AZURE_TOKEN_CREDENTIALS	ManagedIdentityCredential en Azure; AzureCliCredential sólo en desarrollo local.

Variable / propiedad	Valor o propósito
FABRIC_GRAPHQL_ENDPOINT	Endpoint del servicio de datos entregado por DE. Se configura por ambiente; su valor privado no se reproduce en este documento.
AzureWebJobsStorage	Conexión a la cuenta de almacenamiento del host. En Azure no debe contener UseDevelopmentStorage=true.
APPLICATIONINSIGHTS_CONNECTION_STRING	Conexión al Application Insights asociado. No incluir su valor en documentación o repositorios.
PYTHON_APPLICATIONINSIGHTS_ENABLE_TELEMETRY	false. Evita agregar otro mecanismo de instrumentación al configurado.
PYTHON_ENABLE_OPENTELEMETRY	false. Se usan spans manuales y el exportador explícito del proyecto.
functionAppConfig.runtime	name=python; version=3.12.
functionAppConfig.deployment.storage	Contenedor deployments y autenticación StorageAccountConnectionString usando AzureWebJobsStorage.

La aplicación detecta el entorno Azure mediante los indicadores del worker y rechaza AzureCliCredential si se encuentra alojada allí. Para la identidad asignada por el sistema no se configura AZURE_CLIENT_ID.

La plantilla de ejemplo deploy/azure-settings.example.json contiene marcadores de referencia. Los valores de credenciales se administran en Azure; los ejemplos de este documento utilizan variables y no claves reales.

Infraestructura como código

Archivo	Uso
deploy/azuredeploy.json	Declara plan, Function, Storage, contenedor, Insights, workspace, identidad, configuración y políticas de publicación.
deploy/deploy.ps1	Valida, aprovisiona o publica según el parámetro Action.
scripts/package_function.py	Construye dist/esj-function.zip con una lista explícita de fuentes permitidas.
.funcignore / .gitignore	Exclusiones para configuración local, herramientas, datos, pruebas y archivos temporales según su uso.

Los nombres de la Function y Storage incluyen un sufijo determinista derivado del ID del grupo. La plantilla usa un parámetro securestring para el endpoint de datos y no devuelve claves en sus outputs.

El script provision aplica ARM en modo incremental. Una nueva ejecución restaura la configuración declarada, incluidos los app settings, por lo que deben revisarse los cambios manuales antes de reaplicar la plantilla.

Preparación local

Requisitos del puesto de desarrollo: Python 3.12, Azure CLI, Azure Functions Core Tools v4 y Azurite. Ejecutar desde la raíz del proyecto:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Para probar localmente, copiar local.settings.example.json a local.settings.json y configurar el endpoint de datos y AzureCliCredential. Iniciar sesión con la cuenta del tenant del proyecto mediante az login --allow-no-subscriptions. Si Azure CLI muestra un selector interactivo de suscripciones, completar la selección antes de escribir otro comando.

Iniciar Azurite en una terminal y func start en otra. El endpoint local es el indicado en el apartado API. No trasladar el entorno virtual de Windows al host Linux.

Validación y creación de infraestructura

```
.\deploy\deploy.ps1 -Action validate
.\deploy\deploy.ps1 -Action provision
```

validate ejecuta la validación ARM sin crear recursos. provision valida y aplica la infraestructura. El script toma el endpoint del archivo local, crea un archivo temporal de parámetros dentro de dist y lo elimina al finalizar. Antes de ejecutarlo, verificar la suscripción y el grupo indicados en sus parámetros.

Publicación del código

```
.\deploy\deploy.ps1 -Action publish
```

publish obtiene el nombre de la Function desde el despliegue ARM, regenera el ZIP y ejecuta az functionapp deployment source config-zip con --build-remote true. El build remoto instala las dependencias Linux de requirements.txt.

El ZIP incluye function_app.py, host.json, requirements.txt y el paquete esj_api con su consulta. Excluye local.settings.json, .venv, datos de diagnóstico, documentación y herramientas de trabajo. No es un paquete autónomo con dependencias ya instaladas.

El despliegue documentado es manual mediante scripts. No hay un pipeline CI/CD ni un mecanismo automático de rollback incluido en esta implementación.

Comprobación posterior y recuperación

```
.\.venv\Scripts\python.exe -m scripts.check_azure
.\deploy\check-telemetry.ps1
```

- Comprobar el estado de la Function, el runtime y que la identidad del sistema esté habilitada.
- Verificar 401 sin clave o con clave inválida y 400 ante una solicitud propia inválida.
- Exigir HTTP 200 con las nueve colecciones para dar por terminada la validación de datos desde Azure.
- Comprobar que X-Request-ID y X-Trace-ID permitan ubicar los eventos y dependencias de la solicitud.

- Si una publicación introduce un problema, conservar la configuración válida y republicar una versión anterior conocida mediante el mismo build remoto. Mantener el código y sus artefactos identificados antes de cada cambio.

check_azure obtiene la clave sólo en memoria y guarda estados y correlación en dist/azure-verification.json. Devuelve código de salida distinto de cero si algún resultado no coincide con el esperado; un 503 no debe interpretarse como integración aprobada.

Application Insights, logs y trazabilidad

La instrumentación tiene dos recorridos coordinados: el host Azure Functions envía solicitudes y logs Python; el worker envía spans manuales mediante AzureMonitorTraceExporter. No se instala una segunda exportación de logs ni una autoinstrumentación de las llamadas HTTP.

Identificador	Propósito
request_id	Identificador generado por la API y entregado al consumidor.
invocation_id	Ejecución concreta del host Functions.
trace_id	Traza W3C; coincide con operation_Id en Application Insights.
span_id	Operación específica dentro de la traza.

El contexto recibido del host es el padre de los spans. Se propaga traceparent al llamar al servicio externo. Esto permite seguir lo instrumentado por el backend; no supone acceso a la telemetría interna de la fuente de datos.

Eventos / operaciones	Información registrada
request.started / completed	Inicio, ruta fija, estado HTTP y duración.
request.rejected / failed	Código de validación o fallo y diagnóstico interno controlado.
operation.started / completed / failed	Inicio, duración y resultado de la obtención del token y de la consulta.
fabric.response / fabric.results	Estado HTTP del origen y cantidades recibidas por colección. Son nombres técnicos del cliente existente.
esj.request / fabric.token / fabric.graphql	Spans de la solicitud, token y dependencia HTTP, correlacionados con la invocación.

Los logs propios no contienen tokens, claves, cuerpos del proveedor, suministros, coordenadas ni descripciones. Las excepciones externas no se copian al consumidor ni se exportan automáticamente con su mensaje completo. El host puede registrar la URL HTTP de entrada, por lo que tampoco deben enviarse datos sensibles en sus parámetros.

Configuración de monitoreo

Elemento	Configuración de desarrollo
Niveles	Eventos propios, Function, Worker, Host.Results y Host.Aggregator en Information; resto en Warning.

Elemento	Configuración de desarrollo
Muestreo del host	Desactivado para la validación inicial. Los spans respetan la decisión del padre W3C y se registran sin padre.
HTTP extendido	Habilitado para registrar resultado HTTP y correlación entrante.
Dependencias automáticas	Deshabilitadas para evitar duplicación con los spans manuales.
Retención / cuota	Log Analytics: 30 días y cuota diaria configurada de 1 GB.
Exportación de spans	En lotes y con buffer en memoria; sin persistencia de telemetría en disco.
Alertas y tableros	No declarados en la infraestructura actual; la consulta operativa se realiza mediante Logs y KQL.

Interpretación de fallos. Una invocación manejada puede figurar como success=true en el host y devolver HTTP 503. Para medir errores de la API usar el código HTTP, request.completed y la dependencia fallida; no basarse sólo en el éxito de ejecución del host.

La ingesta puede demorar varios minutos. Una cuota de ingesta alcanzada o la pérdida de un buffer por cierre del proceso puede reducir la visibilidad. La cuota configurada no representa un presupuesto monetario estricto.

Consultas operativas

En el portal Azure abrir appi-pi-epre-dev → Logs. Para las siguientes consultas usar el contexto de Application Insights, con tablas requests, traces y dependencies. En el workspace Log Analytics los nombres equivalentes son AppRequests, AppTraces y AppDependencies.

```
let requestId = "REEMPLAZAR_X_REQUEST_ID";
traces
| where timestamp > ago(24h)
| extend e = parse_json(message)
| where tostring(e.request_id) == requestId
| project timestamp, operation_Id, event=tostring(e.event),
          status=toint(e.status_code), reason=tostring(e.reason)
| order by timestamp asc
let traceId = "REEMPLAZAR_X_TRACE_ID";
union withsource=tabla requests, dependencies, traces
| where timestamp > ago(24h) and operation_Id == traceId
| order by timestamp asc
```

El archivo docs/application-insights.kql incluye también tasa de errores HTTP, latencia p95 y dependencias fallidas. deploy/check-telemetry.ps1 descarga únicamente metadatos seleccionados en dist/azure-telemetry.json.

Información de soporte del proyecto

Procedimiento de atención

- Registrar fecha, ambiente, ruta y estado HTTP observado.
- Solicitar X-Request-ID y X-Trace-ID de la respuesta, si están disponibles.
- Buscar la solicitud en Application Insights y revisar el último evento propio y las dependencias.
- Distinguir un rechazo del host, un fallo del backend y un rechazo o problema del servicio de datos.
- Escalar al responsable correspondiente con los identificadores y un diagnóstico sin credenciales ni datos de negocio.

Situación	Primera revisión	Escalamiento
No hay respuesta o ruta desconocida	Disponibilidad de la Function, URL, ruta y método.	Backend / Azure.
401 de Azure	Presencia y vigencia de x-functions-key.	Administrador de la Function.
400 de la aplicación	Body vacío y ausencia de filtros no admitidos.	Consumidor / backend.
503	Evento request.failed: configuración, token o rechazo del servicio de datos.	Backend / Azure; responsable DE cuando el origen rechaza el acceso.
502	Conectividad, respuesta, colecciones y campos; buscar el diagnóstico controlado.	Backend y, si corresponde, equipo DE.
504	Latencia y estado de la dependencia externa.	Backend / operación y responsable del origen.
500	Tipo y ubicaciones del stack registradas por el backend.	Equipo de desarrollo.

Preguntas frecuentes

¿Qué debo enviar en el body? Nada. Es un GET con Body → none y sin parámetros de datos.

¿Qué clave utiliza el consumidor? Una Function Key en el header x-functions-key. Esa clave autoriza la entrada HTTP y no se envía al servicio de origen.

¿La Function necesita una sesión personal abierta? En Azure utiliza su identidad administrada. La sesión Azure CLI se utiliza exclusivamente en desarrollo local.

¿Dónde están los cálculos de indicadores? La API no calcula indicadores. Sus definiciones y preparación corresponden al apartado DE.

¿Cada cuánto se actualiza la API? La API consulta la fuente con cada solicitud válida. La actualización de los datos depende del servicio de origen; no hay un refresco programado en este backend.

¿La cantidad recibida es el total disponible? No. cantidad_devuelta cuenta los items incluidos. La consulta actual no recorre páginas adicionales.

¿Por qué puede pasar la prueba unitaria y fallar Azure? Las pruebas automatizadas validan el código con respuestas controladas. La integración real depende además de configuración, disponibilidad y autorización de la identidad del servicio.

¿Qué datos debo compartir ante un error? Ambiente, estado HTTP, fecha y los identificadores de solicitud y traza. No compartir claves, tokens ni respuestas completas con datos de negocio.

Errores comunes o conocidos

- Enviar una consulta JSON en el body del GET produce 400.
- Enviar filtros desde, hasta o region produce 400 porque no están implementados.
- Probar localmente sin clave no valida la seguridad del host en Azure.
- Configurar AzureCliCredential en el entorno Azure es rechazado por la aplicación.
- Una identidad habilitada en Azure puede seguir sin autorización para consultar una dependencia externa.
- Una respuesta parcial o con un campo faltante se rechaza completa: el backend no genera valores de reemplazo.
- El estado Succeeded del host no equivale a una respuesta HTTP exitosa de la API.

Observaciones y pendientes de cierre

Pendiente	**Responsable de definición / validación**
Actualizar y aprobar el apartado DE	Analista de Ingeniería de Datos.
Aprobar el JSON público, campos, tipos y necesidades del consumidor	Consumidor EPRE con backend y DE.
Completar la prueba HTTP 200 desde Azure con la identidad del servicio	Backend / operación, con autorización del servicio de datos.
Definir filtros, orden, paginación y volumen esperado	Consumidor, backend y DE. La implementación actual no los incorpora.
Definir ambiente productivo, operación, alertas y criterios de capacidad	Responsables del proyecto y de Azure.
Completar equipo, contactos y aprobación del documento	Coordinación del proyecto.

Los pendientes de aceptación no deben presentarse como funciones ya implementadas. Ante cambios en la consulta o el contrato, actualizar las pruebas y esta documentación antes de publicar una nueva versión.

Términos comunes

Término	Definición aplicada al proyecto
API REST / endpoint	Punto de acceso HTTP publicado para consultar información.
Azure Function	Código ejecutado por Azure Functions ante una solicitud HTTP.
Function Key	Clave de entrada validada por Azure Functions.
Managed Identity	Identidad administrada por Azure, asociada a la Function para autenticación de servicio.
Object ID / Principal ID	Identificador del objeto de la identidad en Microsoft Entra.
Application / Client ID	Identificador de aplicación de la representación de servicio.
App settings	Configuración por ambiente disponible para el proceso de la Function.
Traza / span	Recorrido correlacionado de una solicitud y sus operaciones individuales.
Application Insights / Log Analytics	Servicios para registrar, consultar y conservar la telemetría.
ARM / build remoto	Infraestructura declarativa de Azure / compilación e instalación de dependencias en el entorno Linux de publicación.

Referencias del backend y Azure

Fuentes internas verificadas: function_app.py, esj_api/, host.json, requirements.txt, tests/, scripts/, deploy/azuredeploy.json, deploy/deploy.ps1 y los reportes de diagnóstico. Las claves y los valores privados de conexión no forman parte de este documento.

Referencia	Ubicación
Contrato y desarrollo local	README.md
Recursos y verificaciones Azure	docs/azure-dev.md
Configuración de observabilidad	docs/azure-functions.md
Consultas de soporte	docs/application-insights.kql
Último diagnóstico HTTP	dist/azure-verification.json; sólo metadatos.
Azure Functions Python	https://learn.microsoft.com/azure/azure-functions/functions-reference-python
Flex Consumption y despliegue	https://learn.microsoft.com/azure/azure-functions/flex-consumption-how-to
Configuración del host	https://learn.microsoft.com/azure/azure-functions/functions-host-json
Identidades administradas	https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/overview